The Evolution of C++: C++14 through C++23

A five-session course for C++98/C++11 engineers

Format: 5 sessions, 2 hours each (10 contact hours) **Audience:** Working software engineers fluent in C++98 through C++11 **Toolchain:** GCC 14+ or Clang 18+ with `-std=c++23`; Compiler Explorer for live demos **Session shape:** Session 1: ~100 minutes of lecture with live demos and a 20-minute guided exercise. Sessions 2 to 5: four blocks, each about 20 minutes of lecture followed by a 10-minute lab on Compiler Explorer (one file, three TODOs on what was just taught); the cumulative telemetry exercise is at-home work

---

1. Course design rationale

Why not one session per standard?

The four revisions covered here are wildly uneven in size and in importance to a working engineer:

Standard	Character	Rough share of "must know" material
C++14	Bug-fix and polish release for C++11	~5%
C++17	Medium release; the "everyday" features people use constantly	~25%
C++20	The largest change since C++11; four major language features plus a large library	~50%
C++23	Medium release; completes C++20 and fills library gaps	~20%

A one-session-per-standard layout would give C++14 far too much time and C++20 far too little. It would also scatter the most important stories across sessions. Constant-expression evaluation, for example, was relaxed in C++14, extended in C++17, transformed in C++20, and loosened again in C++23. Lambdas gained generic parameters in C++14, `constexpr` in C++17, and template parameters in C++20. Ranges arrived in C++20 and became practical in C++23. Teaching these chronologically means teaching each one four times.

The approach used here

The course is organized **thematically**, by the problem each group of features solves, and every feature is tagged with the standard that introduced it. This lets attendees learn the language as it is actually used while still being able to answer "is this available under `-std=c++17`?" A version timeline (Appendix A) is distributed as a handout so the chronology is never lost.

The five sessions move from the smallest, safest changes to the largest and most disruptive:

1. **The everyday language**: small syntax that changes how ordinary code is written
2. **Vocabulary types and the library**: the types that replace raw pointers, sentinels, and out-parameters
3. **Compile-time and generic programming**: `constexpr`, concepts, and the death of SFINAE
4. **Ranges and algorithms**: a new model of iteration
5. **Concurrency, coroutines, modules, and adoption**: the big-ticket items and a migration plan

Each session answers three questions for the audience: what does this replace from C++98/11, why is it better, and what is the first thing I should change in my own codebase.

Learning objectives

By the end of the course, attendees will be able to:

- Read and write idiomatic C++20/23 code and recognize modern idioms in code review
- Identify C++98/11 patterns in existing code that have a strictly better modern replacement
- Choose the correct vocabulary type (`optional`, `variant`, `expected`, `string_view`, `span`) for an interface
- Write constrained templates with concepts instead of SFINAE
- Compose ranges pipelines and use the constrained algorithms
- Explain the tradeoffs of coroutines and modules well enough to decide whether to adopt them
- Produce a prioritized modernization plan for a legacy C++11 codebase

Prerequisites and pre-work

Attendees should be comfortable with C++11: `auto`, range-based `for`, lambdas, rvalue references and move semantics, `std::unique_ptr/std::shared_ptr`, `nullptr`, `override`, and basic templates. Session 1 opens with a 20-minute calibration segment that fills the most common C++11 gaps.

Before Session 1, attendees should:

- Confirm they can compile with `g++ -std=c++23` or `clang++ -std=c++23`
- Bookmark cppreference.com and the compiler support table at cppreference's "C++ compiler support" page
- Open godbolt.org and verify they can run a hello-world

---

2. Session schedule

Session 1: The Everyday Language

Theme: Small syntax changes that touch nearly every function you write. **Standards covered:** C++14 (nearly all of it), C++17 core syntax, C++20 and C++23 small features.

Time	Segment	Content
0:00	Course overview	Why thematic rather than chronological; the timeline handout; how to read the compiler support table
0:10	C++11 calibration	Quick check of the features this course assumes: `auto`, move semantics, lambdas, smart pointers, `constexpr` (C++11 form), `enum class`, uniform initialization. Common misconceptions: when `std::move` does nothing, `auto` and references, capture by reference in escaping lambdas
0:30	C++14: the polish release	Generic lambdas (`auto` parameters); lambda init-capture `[p = std::move(p)]`; return type deduction for functions; `decltype(auto)`; `std::make_unique`; binary literals and digit separators; `[[deprecated]]`; standard user-defined literals (`"abc"s`, `10ms`); `std::exchange`; `std::integer_sequence`; relaxed `constexpr` (preview, covered fully in Session 3); removal of `gets`
0:55	C++17: syntax you will use daily	Structured bindings; `if` and `switch` with initializer; `inline` variables (header-only globals at last); nested namespace definitions; `[[nodiscard]]`, `[[maybe_unused]]`, `[[fallthrough]]`; guaranteed copy elision and what it means for factories and non-movable types; class template argument deduction (CTAD); `std::string_view` (preview); stricter expression evaluation order; `noexcept` as part of the function type; removals: `auto_ptr`, `register`, trigraphs, dynamic exception specifications, `random_shuffle`
1:20	C++20 and C++23: small but valuable	Designated initializers; range-for with initializer; three-way comparison `<=>` and defaulted comparisons (a full section: how `==` and `<=>` are rewritten, why you write two lines instead of six functions); `using enum`; `[[likely]]`/`[[unlikely]]`; `[[no_unique_address]]`; `char8_t`; `__VA_OPT__`. C++23: `auto(x)` decay copy; `size_t` literal `uz`; `#elifdef`, `#warning`; `std::to_underlying`; `std::unreachable`; `[[assume]]`; labels at end of compound statements
1:35	Guided exercise	Modernize a 150-line C++11 file: replace `typedef`/pair-based returns with structured bindings, add `[[nodiscard]]` to value-returning functions, replace

Time	Segment	Content
		hand-written comparison operators with <=>, convert `std::pair<iterator,bool>` idioms to `if`-with-initializer
1:55	Wrap-up	Preview of Session 2; "one thing to change Monday morning"

Key takeaways: Almost every line of a modern function looks slightly different from its C++11 form, and each difference removes a class of bug (unused results, forgotten `break`, dangling globals in headers, six-way comparison boilerplate).

Session 2: Vocabulary Types and the Standard Library

Theme: The types that replace raw pointers, sentinel values, out-parameters, `char*/length` pairs, and `printf`. **Standards covered:** C++17 library, C++20 library, C++23 library.

Time	Segment	Content
0:00	Recap and framing	The "vocabulary type" idea: types that appear in interfaces so callers and callees agree on meaning without documentation
0:06	Views over data (non-owning)	`std::string_view` (C++17) and `std::span` (C++20): what they replace (`const char*` + length, `const std::vector<T>&` when you did not need a vector), lifetime rules and the dangling-view trap, when to still take `const std::string&`. C++23: `std::string::contains`, `std::mdspan` for multidimensional views (preview)
0:20	**Lab** (10 min)	`labs/s02/lab1_views.cpp` on Compiler Explorer: `const std::string&` to `string_view`, pointer-plus-length to `span`, a dangling view fixed
0:30	Maybe, either, and anything	`std::optional` (C++17) replacing sentinel values and `bool` out-parameters; C++23 monadic operations (`and_then`, `transform`, `or_else`). `std::variant` and `std::visit` (C++17) replacing tagged unions and `void*`; the overload-set visitor idiom. `std::any` and why you will rarely want it
0:50	**Lab** (10 min)	`lab2_optional_variant.cpp`: `bool` + out-parameter to `optional`, hand unwrapping to `transform`, a tagged union to `variant` + `visit`
1:00	Error handling: `std::expected`	`std::expected<T, E>` (C++23): typed error returns that compose; the monadic interface; when to prefer it over exceptions and over error codes; how it interacts with `[[nodiscard]]`. Short discussion of exceptions in

Time	Segment	Content
		embedded and safety-critical contexts
1:10	Formatting and output	`std::format` (C++20) and `std::print/std::println` (C++23): type-safe, compile-time-checked format strings; formatting user-defined types with `std::formatter`; formatting ranges (C++23); what to do with `printf` and `iostream` code
1:20	**Lab** (10 min)	`lab3_expected_format.cpp`: out-parameters to `expected`, hand unwrapping to `and_then`, a `std::formatter`
1:30	Library tour (rapid)	C++17: `std::filesystem`, `std::byte`, `to_chars/from_chars`, `std::invoke`, `std::apply`, `std::clamp`, `std::gcd`, map `try_emplace/insert_or_assign/extract/merge`, `std::size/std::data`. C++20: `std::bit_cast` and `<bit>`, `std::source_location`, `std::erase_if`, `starts_with/ends_with`, `contains` for associative containers, `<chrono>` calendars and time zones, `std::numbers`, `std::midpoint`, `std::lerp`, `std::to_array`. C++23: `std::flat_map/std::flat_set`, `std::stacktrace`, `std::move_only_function`, `std::byteswap`, `std::out_ptr`, `std::forward_like`, `resize_and_overwrite`
1:40	Interface design checklist	Which vocabulary type for which situation
1:42	**Lab** (10 min)	`lab4_interface_checklist.cpp`: three C++11 signatures rewritten with the checklist (`expected`, `span` + `optional`, `optional` of a struct)
1:52	Wrap-up	The telemetry exercise, launched as at-home work: `parse_record` to `std::expected`, `find_sensor` to `std::optional`, the report to `std::println`, then `formatter`, `string_view`, `span`, monadic ops

Key takeaways: A modern interface says what it means. `optional` means "maybe", `expected` means "or this error", `string_view` means "I will only look", `span` means "a contiguous run I do not own".

Session 3: Compile-Time and Generic Programming

Theme: Moving work to compile time and writing templates a human can read. **Standards covered:** The `constexpr` thread through C++14/17/20/23; C++17 template features; C++20 concepts; C++23 deducing `this`.

Time	Segment	Content
0:00	Recap and framing	Why compile-time programming matters for embedded and high-assurance code: tables, checks, and configuration with zero runtime cost and no initialization-order problems
0:06	The `constexpr` story	C++11's single-return-statement functions. C++14: loops, locals, and mutation allowed. C++17: `constexpr` lambdas, `if constexpr`, `constexpr std::array` usability. C++20: `constexpr` dynamic allocation, `std::vector` and `std::string` at compile time, `constexpr` virtual functions and `try/catch`, `consteval` (immediate functions), `constinit` (guaranteed static initialization), `std::is_constant_evaluated`. C++23: `if consteval`, non-literal variables and `static/thread_local` in `constexpr` functions, `constexpr std::unique_ptr`, `static operator()`. Demo: a compile-time CRC table and a compile-time-validated configuration struct
0:20	**Lab** (10 min)	`labs/s03/lab1_constexpr.cpp`: a CRC table made `constexpr`, a `consteval` config check, a parser that works in both worlds, each proven by a `static_assert`
0:30	C++17 template quality of life	Fold expressions (replacing recursive variadic templates); `if constexpr` replacing tag dispatch and overload tricks; `auto` non-type template parameters; CTAD and deduction guides; variable templates (C++14) and `_v`/`_t` trait aliases
0:45	**Lab** (10 min)	`lab2_templates17.cpp`: recursion to fold expressions, tag dispatch to `if constexpr`, a CTAD deduction guide
1:00	C++20 concepts	The problem: `enable_if` and SFINAE error messages. `requires` clauses, `requires` expressions, named concepts, the standard concepts library (`std::integral`, `std::invocable`, `std::ranges::range`, and friends). Abbreviated function templates (`void f(std::integral auto x)`). Constrained `auto`. Subsumption and overload selection by constraint. Demo: convert an `enable_if` overload set to concepts and compare the diagnostics side by side
1:20	**Lab** (10 min)	`lab3_concepts.cpp`: a `requires`-expression concept, a constrained `to_json` and its error message before and after, a second overload for ranges
1:30	Deducing `this` (C++23), and reading diagnostics	Explicit object parameters: one member instead of `const`/non-`const` pairs, recursive lambdas, CRTP without the template parameter; how to read a concepts error
1:45	**Lab** (10 min)	`lab4_deducing_this.cpp`: the three deducing-`this` idioms, written
1:55	Wrap-up	What to reach for: `constexpr` by default, `consteval`

Time	Segment	Content
		when it must be compile time, concepts on every public template; the telemetry exercise (CRC table, config validation, constrained `serialize`) launched as at-home work

Key takeaways: Templates are no longer a specialist skill. Concepts and `if constexpr` make generic code readable and its errors understandable, and `constexpr` makes the compiler do work you used to script or hand-compute.

Session 4: Ranges and Algorithms

Theme: A new model of iteration that composes. **Standards covered:** C++17 parallel algorithms, C++20 ranges, C++23 ranges completion.

Time	Segment	Content
0:00	Recap and framing	The iterator-pair problem: verbosity, mismatched pairs, no composition, no lazy evaluation
0:06	The C++20 ranges foundation	Range concepts (`input_range` through `contiguous_range`); sentinels replacing end iterators; `std::ranges::begin/end` customization point objects; the constrained algorithms in `std::ranges::` (why they exist alongside `std::`); projections (the feature people miss most)
0:20	**Lab** (10 min)	`labs/s04/lab1_constrained_algorithms.cpp`: `sort`, `find` and `max_element` with projections; `std::ranges::dangling` met on purpose
0:30	Views and pipelines	Lazy evaluation; `std::views::filter`, `transform`, `take`, `drop`, `reverse`, `iota`, `split`, `join`, `keys/values`; the pipe syntax; view semantics and lifetime (`borrowed_range`, the dangling iterator problem, `std::ranges::dangling`); why `const` views are tricky; cost model and when a plain loop is still right
0:50	**Lab** (10 min)	`lab2_views.cpp`: a lazy `filter | transform | take` pipeline, `ranges::to`, the `const` view trap
1:00	C++23: ranges become practical	`std::ranges::to` (finally materializing a pipeline into a container); `views::zip`, `enumerate`, `chunk`, `chunk_by`, `slide`, `stride`, `adjacent`, `join_with`, `cartesian_product`, `repeat`, `as_rvalue`; `ranges::fold_left` and friends; `ranges::contains`, `starts_with`, `ends_with`, `find_last`; formatting ranges with `std::print`. Demo: the same data-processing task in C++11, C++20, and C++23
1:13	Parallel algorithms	C++17 execution policies (`seq`, `par`, `par_unseq`, C++20

Time	Segment	Content
		`unseq`); what actually parallelizes on GCC/libstdc++ and Clang/libc++ (TBB dependency); data-race responsibilities; when it is worth it
1:20	**Lab** (10 min)	`lab3_cpp23_ranges.cpp`: `chunk_by` as group-by, `fold_left`, `enumerate`
1:30	Algorithm additions worth knowing	C++17: `std::sample`, `std::reduce`, `transform_reduce`, `inclusive_scan`, `for_each_n`, `search` with searchers. C++20: `std::shift_left/shift_right`, `std::lexicographical_compare_three_way`, `std::ranges::` versions of everything. C++23: `ranges::iota`, `ranges::shift_left`, `std::ranges::fold_*`
1:37	Guidance, and the same task three ways	Ranges in production code; one task in C++11, C++17 and C++20/23
1:42	**Lab** (10 min)	`lab4_three_ways.cpp`: attendees write the C++17 and the ranges version themselves; `main` checks all three agree
1:52	Wrap-up	The telemetry exercise (hand-written loops to constrained algorithms and views, report byte-identical) launched as at-home work

Key takeaways: Ranges are the biggest change to how loops are written since range-based `for`. C++20 provides the model; C++23 provides the pieces that make it usable day to day.

Session 5: Concurrency, Coroutines, Modules, and Adoption

Theme: The large-scale features, and how to bring a legacy codebase forward. **Standards covered:** C++14/17/20/23 concurrency, C++20 coroutines, C++20/23 modules.

Time	Segment	Content
0:00	Recap and framing	The three "big four" features not yet covered (concepts and ranges were Sessions 3 and 4) and why they are last: they change build systems, control flow, and program structure, not just syntax
0:05	Concurrency, C++14 to C++23	C++14: `shared_timed_mutex`. C++17: `shared_mutex`, `scoped_lock` (deadlock-free multi-lock), `hardware_destructive_interference_size`. C++20: `std::jthread` and cooperative cancellation with `stop_token/stop_source`; `std::latch`, `std::barrier`, `std::counting_semaphore`; `atomic::wait/notify`; `std::atomic_ref`; `std::atomic<std::shared_ptr>`;

Time	Segment	Content
		`std::osyncstream` for sane multi-threaded logging. C++23: `std::move_only_function` for task queues. What is still missing (executors) and where it is headed
0:20	**Lab** (10 min)	`labs/s05/lab1_jthread_stop_token.cpp`: `thread` + flag to `jthread` + `stop_token`, two `lock_guard`s to `scoped_lock`, a spin-wait to `binary_semaphore`
0:30	Coroutines	The C++20 language feature vs. the (absent in C++20) library: `co_await`, `co_yield`, `co_return`; promise types and awaiters at a high level; why you should not hand-write a coroutine type in production; `std::generator` (C++23) as the first standard coroutine type; where coroutines pay off (async I/O, state machines, lazy sequences) and where they do not (hot loops, hard real-time). Demo: a `std::generator`-based lazy parser
0:50	**Lab** (10 min)	`lab2_generator.cpp`: an eager vector to `std::generator`, a lazy parser, what happens to a suspended coroutine
1:00	Modules	What modules fix (header parsing cost, macro leakage, ODR fragility) and what they do not. `export module`, `import`, module partitions, header units. `import std;` (C++23). The honest current state of build system and compiler support (CMake 3.28+, GCC, Clang, MSVC) and a recommended adoption posture
1:16	Deprecations and removals to audit for	Cumulative list across C++14/17/20/23: `auto_ptr`, `random_shuffle`, `std::iterator`, `throw()`, `bind1st/bind2nd`, `std::result_of`, `strstream`, `codecvt`, volatile compound operations (C++20, partially restored in C++23), `std::aligned_storage/aligned_union` (C++23), garbage-collection API (C++23). How to find them with `-Wdeprecated-declarations` (on by default) and clang-tidy
1:20	**Lab** (10 min)	`lab3_deprecations.cpp`: a file with six deprecation warnings under `-std=c++23` brought to zero, same output (modules cannot run as one Compiler Explorer file, so this is block 3's lab)
1:30	Adoption roadmap workshop	A prioritized plan for moving a C++11 codebase forward: tier 1 (zero-risk, mechanical: `make_unique`, `[[nodiscard]]`, structured bindings, `string_view` at boundaries, `optional` for sentinels); tier 2 (interface changes: `span`, `expected`, concepts on public templates, `std::format`); tier 3 (architectural: ranges pipelines, `stop_token` cancellation, coroutines, modules); `jthread` itself is a tier-1

Time	Segment	Content
		swap for `thread`. clang-tidy `modernize-*` checks as a force multiplier; compiler flag progression; how to write a coding-standard update. The worksheet is the block's ten-minute lab (1:35): attendees draft a tier-1 list for their own codebase and three volunteers read theirs
1:45	Course close	Reading list, where to follow the language (WG21 papers, cppreference support table), what is coming in C++26 (reflection, contracts, `std::execution`, hardened standard library)

Key takeaways: Coroutines and modules are worth understanding now and adopting deliberately. Concurrency improvements can be adopted immediately. The most valuable thing an engineer can leave with is a short, ordered list of changes to make first.

---

3. Assessment and exercises

Session 1 ends with a 20-minute guided exercise. From Session 2 on, every lecture block is followed by a 10-minute lab done on Compiler Explorer (no local toolchain needed; the same files are in the repo under `labs/` with a finished version beside each starter), four labs per session, fifteen in all, plus the adoption-roadmap worksheet in Session 5. The telemetry exercise is cumulative and done at home: the Session 1 file is the one modernized further in Sessions 2 through 5, so attendees finish with a single C++11 program fully carried to C++23.

An optional capstone for attendees who want it: take a 300 to 500 line module from their own codebase, apply the tier-1 and tier-2 items from the Session 5 roadmap, and submit a before/after diff with a one-paragraph note on what became simpler and what did not.

---

4. Resources

Reference

- cppreference.com, especially the "C++ compiler support" table and the per-standard feature pages
- The C++ Core Guidelines (isocpp.github.io/CppCoreGuidelines)
- Compiler Explorer (godbolt.org)

Books

- Nicolai Josuttis, *C++17: The Complete Guide* and *C++20: The Complete Guide*
- Anthony Williams, *C++ Concurrency in Action*, 2nd edition (covers through C++17; C++20 additions covered in class)
- Rainer Grimm, *C++20: Get the Details* and *C++23: Get the Details*

- Scott Meyers, *Effective Modern C++* (C++11/14; still the best explanation of `auto` and move semantics)

Talks (CppCon, all freely available)

- Bjarne Stroustrup, "The Evolution of C++" and various keynotes on direction
- Timur Doumler, "C++20: The Small Things"
- Andreas Fertig, "C++20 Templates: The Next Level"
- Tristan Brindle, "An Overview of Standard Ranges"
- Herb Sutter's "Welcome Back to C++" talks for the modernization mindset

---

Appendix A: Feature timeline handout

The features taught in this course, by standard. Session numbers in parentheses.

C++14

Generic lambdas (1); lambda init-capture (1); function return type deduction and `decltype(auto)` (1); relaxed `constexpr` (3); variable templates (3); `std::make_unique` (1); binary literals and digit separators (1); `[[deprecated]]` (1); standard literals `s`, `ms`, etc. (1); `std::exchange` (1); `std::integer_sequence` (1); `std::shared_timed_mutex` (5); `gets` removed (1).

C++17

Structured bindings (1); `if`/`switch` with initializer (1); `if constexpr` (3); `inline` variables (1); fold expressions (3); CTAD (1, 3); guaranteed copy elision (1); nested namespaces (1); `[[nodiscard]]`, `[[maybe_unused]]`, `[[fallthrough]]` (1); `constexpr` lambdas (3); `auto` non-type template parameters (3); evaluation order guarantees (1); `noexcept` in the type system (1); `std::optional`, `std::variant`, `std::any` (2); `std::string_view` (2); `std::filesystem` (2); parallel algorithms (4); `std::byte` (2); `to_chars`/`from_chars` (2); `std::invoke`, `std::apply` (2); `std::scoped_lock`, `std::shared_mutex` (5); map `try_emplace`, `extract`, `merge` (2); `std::clamp`, `std::gcd`, `std::sample` (2, 4); removals of `auto_ptr`, `register`, trigraphs, dynamic exception specifications, `random_shuffle` (1, 5).

C++20

Concepts and `requires` (3); ranges (4); coroutines (5); modules (5); three-way comparison `<=>` (1); designated initializers (1); `consteval`, `constinit`, `constexpr` allocation/virtual/`try` (3); abbreviated function templates (3); template lambdas (3); class-type non-type template parameters (3); `using enum` (1); `[[likely]]`, `[[unlikely]]`, `[[no_unique_address]]` (1); range-for with initializer (1); `char8_t` (1); `__VA_OPT__` (1); `std::span` (2); `std::format` (2); `std::jthread`, `stop_token` (5); `std::latch`, `std::barrier`, `std::counting_semaphore` (5); atomic `wait`/`notify`, `std::atomic_ref` (5); `std::osyncstream` (5); `std::bit_cast`, `<bit>` (2); `std::source_location` (2); `<chrono>` calendars and time zones (2); `std::erase_if`, `starts_with`/`ends_with`, `contains` (2); `std::numbers`, `std::midpoint`, `std::lerp`,

`std::to_array` (2); `std::shift_left/shift_right` (4); `std::is_constant_evaluated` (3).

C++23

Deducing `this` (3); `if consteval` (3); `static operator()` and `operator[]`, multidimensional subscript (3); `auto(x)` (1); `uz` literal, `#elifdef`, `#warning` (1); `[[assume]]` (1); relaxed `constexpr` restrictions, `constexpr std::unique_ptr` (3); `std::expected` (2); `std::print/std::println`, range formatting (2); `std::optional` monadic operations (2); `std::flat_map/std::flat_set` (2); `std::mdspan` (2); `std::generator` (5); `std::stacktrace` (2); `std::move_only_function` (2, 5); `std::string::contains` (2); `std::to_underlying`, `std::unreachable` (1); `std::ranges::to` (4); `views::zip`, `enumerate`, `chunk`, `chunk_by`, `slide`, `stride`, `adjacent`, `join_with`, `cartesian_product`, `repeat`, `as_rvalue` (4); `ranges::fold_left` and friends, `ranges::contains`, `starts_with`, `ends_with`, `find_last` (4); `import std;` (5); `std::byteswap`, `std::out_ptr`, `std::forward_like`, `resize_and_overwrite` (2); `std::aligned_storage` deprecated, garbage-collection API removed (5).

Appendix B: Alternative layouts considered

One session per standard (with C++14 folded into Session 1). Rejected because C++20 cannot be covered in two hours and because the `constexpr`, lambda, and ranges stories would each be split across three sessions.

Language first, library second. Two and a half sessions on core language, two and a half on library. Rejected because the most valuable library features (`optional`, `expected`, `span`, ranges) are only understandable alongside the language features they depend on, and because it front-loads the hardest material.

Migration-driven (by legacy pattern). "Here is a C++98 pattern; here is its replacement." Attractive for this audience and used within each session, but as the top-level organization it does not give attendees a mental model of the language, only a lookup table. The thematic layout here keeps the migration framing as the "what it replaces" thread in every segment and as the Session 5 roadmap workshop.